Name of Student: VIKAS FOOLCHAND KUSHWAHA			
Roll Number: 29/B		**Lab Practical Number:** 02	
Title of LAB Practical: Implementing Data preprocessing techniques.			
DOP: 06/11/2026		**DOS:** 16/11/2026	
CO Mapped: CO1	**PO Mapped:** PO1, PO5, PO7, PSO1	**Faculty Signature:**	**Marks:**

AIM:

Implementing Data preprocessing techniques.

Theory:

Data preprocessing is the process of preparing raw data for analysis or machine learning. It improves data quality, consistency, and usefulness.

- **Naming and renaming variables:** Clear, consistent column names make data easier to understand and work with. R's **rename()** changes existing names; **mutate()** can create new variables from existing data.
- **Missing data:** Missing values can lead to errors or biased results. Common approaches include removing incomplete rows or filling missing values using an estimate, such as a mean. The best method depends on why values are missing and how the data will be analyzed.
- **Categorical data:** Categories such as department or grade are often stored as text. Converting them to factors tells R that the values represent distinct groups. Ordered factors are useful when categories have a meaningful ranking, such as low, medium, and high.
- **Data reduction through subsetting:** Subsetting keeps only the rows or columns relevant to a task. This can simplify analysis, reduce processing, and focus attention on the population or variables of interest.

Code:

1. Load necessary packages.

CODE

```
# 1. Install and load necessary packages:
install.packages("dplyr")
install.packages("tidyr")

# The following package provides the piping functionality
# as declared by the %>% symbol.
library(dplyr)

# The following package provides the 'replace_na' function
library(tidyr)
```

OUTPUT

```
** testing if installed package keeps a record of temporary installation path
* DONE (tidyr)

The downloaded source packages are in
        '/tmp/Rtmp543fye/downloaded_packages'
>
> # The following package provides the piping functionality
> # as declared by the %>% symbol.
> library(dplyr)
>
> # The following package provides the 'replace_na' function
> library(tidyr)
>
```

2. Declaring a sample dataset.

CODE

```
# 2. Declare a sample data set
students <- data.frame(
  Student_ID = 1:6,
  Full_Name = c("Ava", "Ben", "Cara", "Dan", "Eva", "Finn"),
  Age = c(20, 22, NA, 19, 21, 23),
  Score = c(95, 82, 71, NA, 32, 58),
  Department = c("Science", "Arts", "Science", NA, "Arts", "Science")
)
students
```

OUTPUT

```
> # 2. Declare a sample data set
> students <- data.frame(
+   Student_ID = 1:6,
+   Full_Name = c("Ava", "Ben", "Cara", "Dan", "Eva", "Finn"),
+   Age = c(20, 22, NA, 19, 21, 23),
+   Score = c(95, 82, 71, NA, 32, 58),
+   Department = c("Science", "Arts", "Science", NA, "Arts", "Science")
+ )
> students
  Student_ID Full_Name Age Score Department
1          1       Ava  20    95    Science
2          2       Ben  22    82       Arts
3          3      Cara  NA    71    Science
4          4       Dan  19    NA       <NA>
5          5       Eva  21    32       Arts
6          6      Finn  23    58    Science
```

3. Naming, renaming and adding variables

CODE

```
# 3. Naming, renaming and adding variable
```

```
students <- students %>%
  rename(
    StudentId = Student_ID,
    Name = Full_Name,
  ) %>%
  mutate(
    # Add a variable
    Passed = if_else(Score >= 40, TRUE, FALSE, missing = NA),
  )

students
```

OUTPUT

```
> # 3. Naming, renaming and adding variable
> students <- students %>%
+   rename(
+     StudentId = Student_ID,
+     Name = Full_Name,
+   ) %>%
+   mutate(
+     # Add a variable
+     Passed = if_else(Score >= 40, TRUE, FALSE, missing = NA),
+   )
> students
  StudentId Name Age Score Department Passed
1         1  Ava  20    95    Science   TRUE
2         2  Ben  22    82       Arts   TRUE
3         3 Cara  NA    71    Science   TRUE
4         4  Dan  19    NA       <NA>     NA
5         5  Eva  21    32       Arts  FALSE
6         6 Finn  23    58    Science   TRUE>
```

4. Dealing with missing data

CODE

```
# 4. Dealing with missing data

# Option A: Remove rows with missing values in selected columns
studentsComplete <- students %>%
  filter(!is.na(Age), !is.na(Score))

studentsComplete

# Option B: Replace missing numeric values with the column mean
studentsFilled <- students %>%
  mutate(
    Age = if_else(is.na(Age), mean(Age, na.rm = TRUE), Age),
    Score = if_else(is.na(Score), mean(Score, na.rm = TRUE), Score)
  )

studentsFilled

# Option C: Replace missing categorical values with "Unknown"
studentsFilled <- studentsFilled %>%
  mutate(Department = tidyr::replace_na(Department, "Unknown"))

studentsFilled
```

OUTPUT

```
> # 4. Dealing with missing data
>
> # Option A: Remove rows with missing values in selected columns
> studentsComplete <- students %>%
+   filter(!is.na(Age), !is.na(Score))
> studentsComplete
  StudentId Name Age Score Department Passed
1         1  Ava  20    95    Science   TRUE
2         2  Ben  22    82       Arts   TRUE
3         5  Eva  21    32       Arts  FALSE
4         6 Finn  23    58    Science   TRUE
>
```

```
> # Option B: Replace missing numeric values with the column mean
> studentsFilled <- students %>%
+   mutate(
+     Age = if_else(is.na(Age), mean(Age, na.rm = TRUE), Age),
+     Score = if_else(is.na(Score), mean(Score, na.rm = TRUE), Score)
+   )
> studentsFilled
  StudentId Name Age Score Department Passed
1         1  Ava  20  95.0    Science   TRUE
2         2  Ben  22  82.0       Arts   TRUE
3         3 Cara  21  71.0    Science   TRUE
4         4  Dan  19  67.6       <NA>     NA
5         5  Eva  21  32.0       Arts  FALSE
6         6 Finn  23  58.0    Science   TRUE
>
```

```
> # Option C: Replace missing categorical values with "Unknown"
> studentsFilled <- studentsFilled %>%
+   mutate(Department = tidyr::replace_na(Department, "Unknown"))
> studentsFilled
  StudentId Name Age Score Department Passed
1         1  Ava  20  95.0    Science   TRUE
2         2  Ben  22  82.0       Arts   TRUE
3         3 Cara  21  71.0    Science   TRUE
4         4  Dan  19  67.6    Unknown     NA
5         5  Eva  21  32.0       Arts  FALSE
6         6 Finn  23  58.0    Science   TRUE
>
```

5. Dealing with categorical data

CODE

```
# 5. Dealing with categorical data

# Convert a character column to a factor
studentsFilled <- studentsFilled %>%
  mutate(
    Department = factor(Department),
    # Set an explicit order for an ordinal category
    Grade = factor(
      case_when(
        Score >= 90 ~ "O",
        Score >= 80 ~ "A",
        Score >= 70 ~ "B",
        Score >= 60 ~ "C",
        Score >= 50 ~ "D",
        Score >= 40 ~ "E",
        TRUE ~ "F"
      ),
      levels = c("F", "E", "D", "C", "B", "A", "O"),
      ordered = TRUE
    )
  )

studentsFilled
```

OUTPUT

```
> # 5. Dealing with categorical data
>
> # Convert a character column to a factor
> studentsFilled <- studentsFilled %>%
+   mutate(
+     Department = factor(Department),
+     # Set an explicit order for an ordinal category
+     Grade = factor(
+       case_when(
+         Score >= 90 ~ "O",
+         Score >= 80 ~ "A",
+         Score >= 70 ~ "B",
+         Score >= 60 ~ "C",
+         Score >= 50 ~ "D",
+         Score >= 40 ~ "E",
+         TRUE ~ "F"
+       ),
+       levels = c("F", "E", "D", "C", "B", "A", "O"),
+       ordered = TRUE
+     )
+   )
> studentsFilled
  StudentId Name Age Score Department Passed Grade
1         1  Ava  20  95.0    Science   TRUE     O
2         2  Ben  22  82.0       Arts   TRUE     A
3         3 Cara  21  71.0    Science   TRUE     B
4         4  Dan  19  67.6    Unknown      NA     C
5         5  Eva  21  32.0       Arts  FALSE     F
6         6 Finn  23  58.0    Science>    TRUE     D
```

6. Data reduction using subsetting

CODE
`# 6. Data reduction using subsetting`

```
# Keep only selected columns and rows meeting a condition
scienceStudents <- studentsFilled %>%
  filter(Department == "Science", Age >= 20) %>%
  select(StudentId, Name, Age, Score, Passed)

scienceStudents
```

OUTPUT

```
> # 6. Data reduction using subsetting
>
> # Keep only selected columns and rows meeting a condition
> scienceStudents <- studentsFilled %>%
+   filter(Department == "Science", Age >= 20) %>%
+   select(StudentId, Name, Age, Score, Passed)
> scienceStudents
  StudentId Name Age Score Passed
1         1  Ava  20    95   TRUE
2         3 Cara  21    71   TRUE
3         6 Finn  23    58   TRUE
>
```

Conclusion:

In today's practical, we learned about important R functions or methods that are used in data preprocessing techniques. Some of them are mentioned below:

- **rename()** was used to change column names.
- **mutate()** was used to create or modify variables.
- **filter()** was used to select rows by filtering the input data.
- **select()** was used to select columns.

For missing data, we chose removal or imputation methods based on the analysis; replacing values with a mean as a simple example. Although, it may not be appropriate for every dataset.